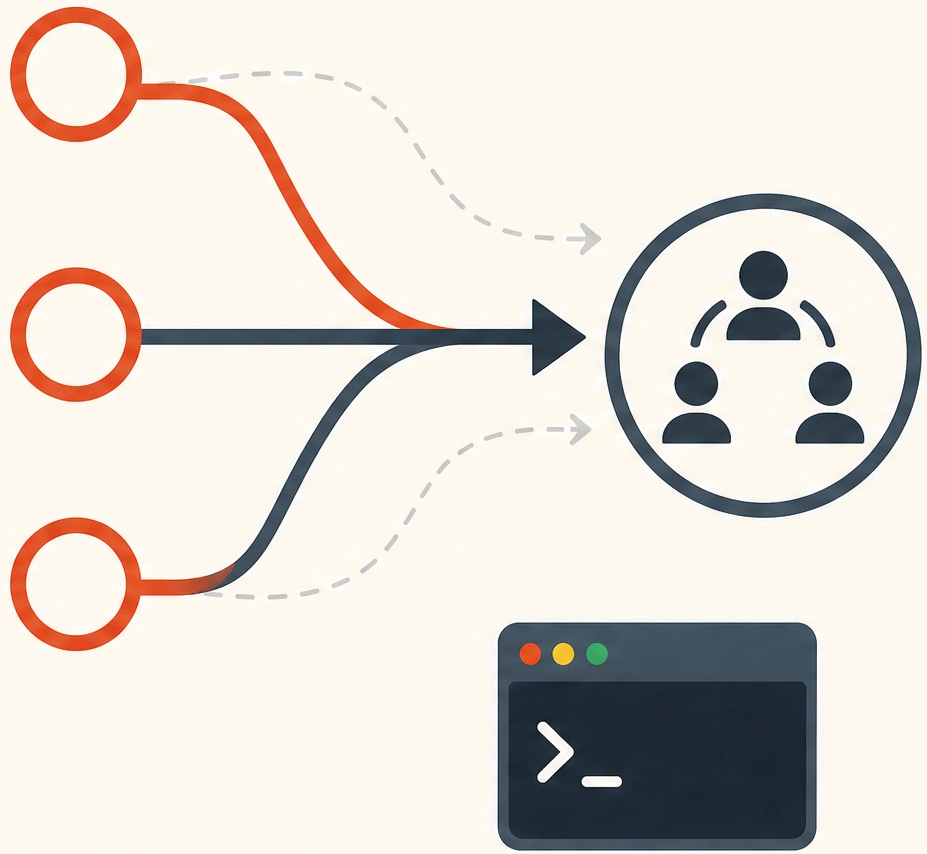
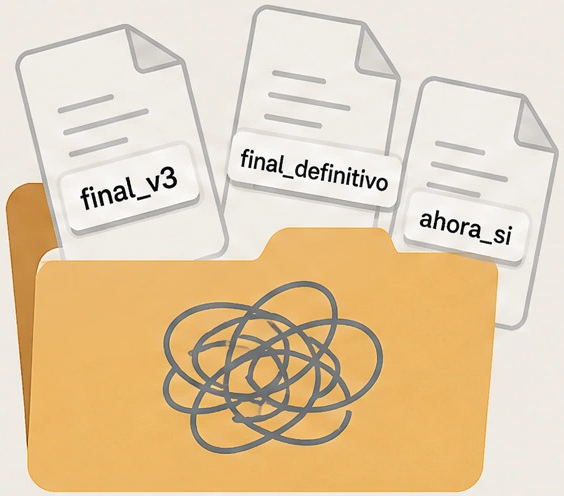
Git + GitHub

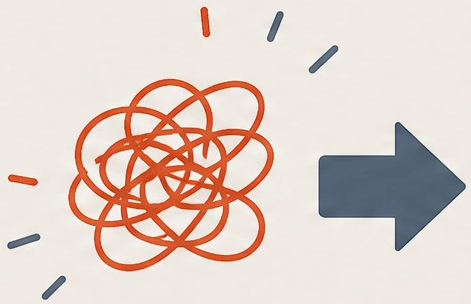
El flujo completo para trabajar con versiones y colaborar

Guía práctica para estudiantes · Jaime Zapata



El problema: cambiar sin romper





- Cambios sobrescritos
- Historial confuso
- Miedo a experimentar

Git convierte el historial en una herramienta de trabajo

Git y GitHub no son lo mismo

clone • fetch • pull • push

Git

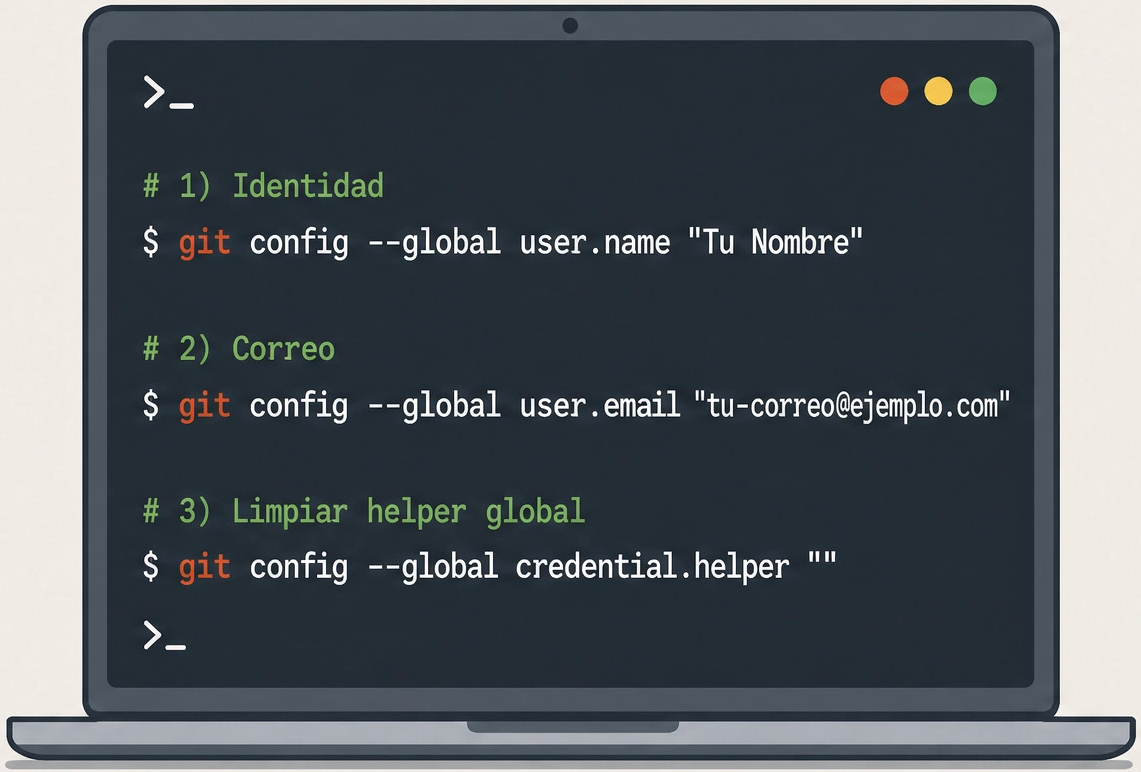
Control de versiones en tu equipo

GitHub

Alojamiento y colaboración

local = trabajo e historial | **remoto** = copia compartida

Configura tu identidad antes de empezar



1 Identidad

```
git config --global user.name "Tu Nombre"
```

firma tus commits con nombre y correo

2 Correo

```
git config --global user.email "tu-correo@ejemplo.com"
```

firma tus commits con nombre y correo

3 Limpiar helper global

```
git config --global credential.helper ""
```

desactiva el ayudante global; no borra por sí solo credenciales guardadas en el sistema

```
git config --global --list
```

comprueba la configuración

Hazlo una vez por equipo

El ciclo local tiene tres estados

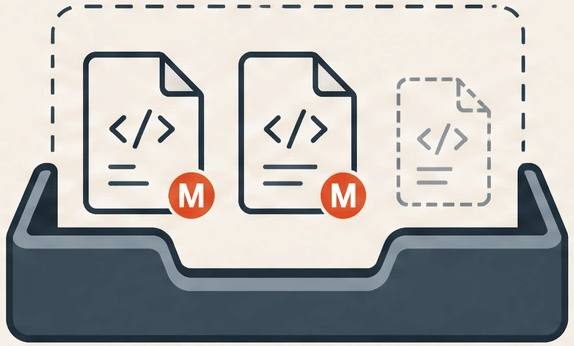
Modified → Staged → Committed

1 Working tree

Editas archivos

Modified

2 Staging area



Seleccionas cambios

```
>_ git add archivo.js
```

3 Repository

Guardas un snapshot

```
>_ git commit -m "mensaje"
```

Observación: usa para ver el estado del ciclo en cualquier momento.

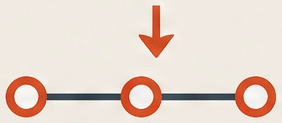
```
>_ git status
```

Del proyecto vacío al repositorio local

1. Proyecto vacío

2. Inicializar y crear el repositorio

```
>_
git init
git add .
git commit -m "chore: inicia proyecto"
```



1. Repositorio alojado

2. Clonar y acceder al proyecto

```
>_
git clone URL_DEL_REPOSITORIO
cd nombre-del-proyecto
```

Repositorio local

Verificación

```
>_ git status
```

```
>_ git log --oneline
```

Registrar cambios: el flujo que repetirás

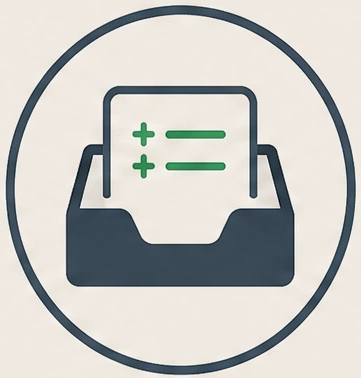
1 Edita

working tree

2 Inspecciona

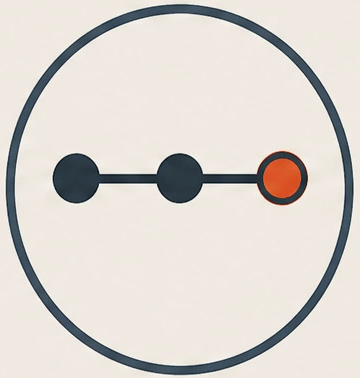
revisa qué cambió

3 Prepara



selecciona el contenido

4 Confirma



crea un commit

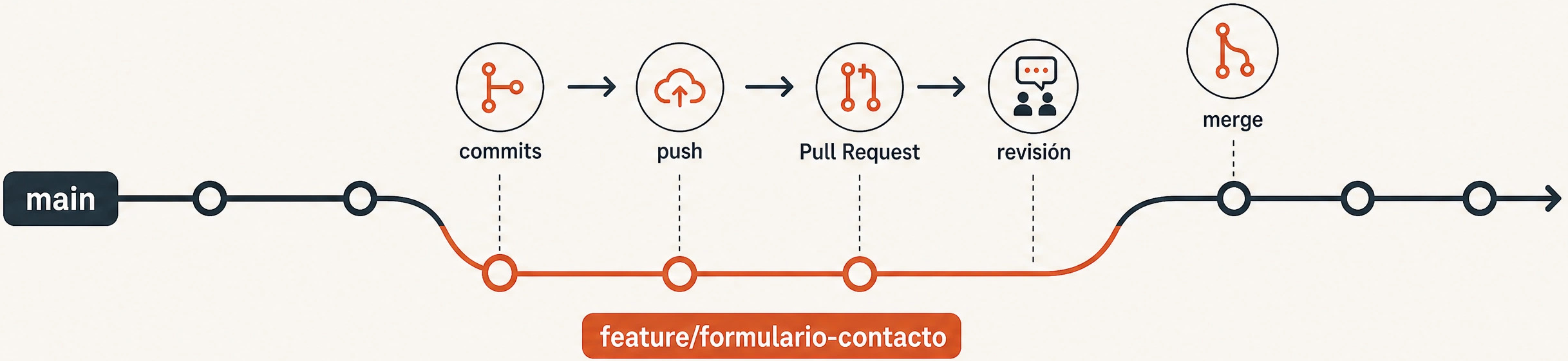
```
>_  `git diff` | `git status` | `git add src/app.js` | `git diff --staged` | `git commit -m "fix: corrige cálculo del total"
```

Un commit = una sola historia

Sincronizar local y remoto


```
>_ | git pull origin main
>_ | git push -u origin feature/login
```

La colaboración ocurre en ramas

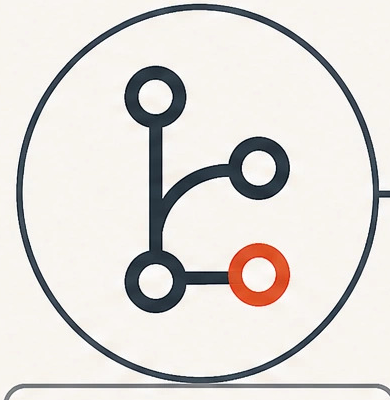



```
$ git switch -c feature/formulario-contacto
$ git add .
$ git commit -m "feat: agrega formulario"
$ git push -u origin feature/formulario-contacto
```

Protege main; experimenta en ramas

Convenciones que reducen fricción

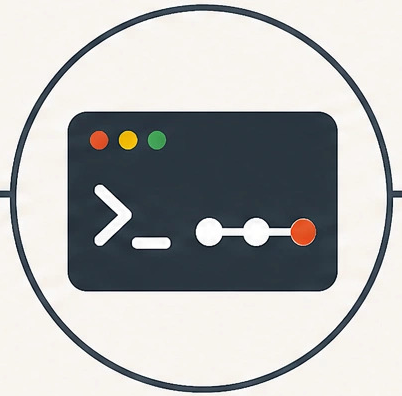
1 Ramas



```
feature/
fix/
docs/
chore/
```

nombres descriptivos

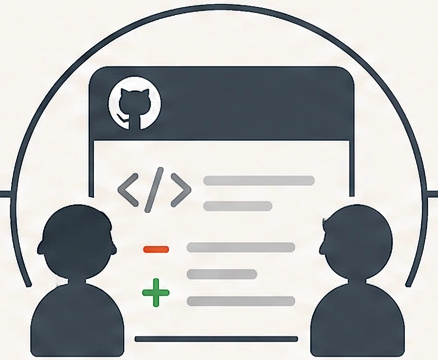
2 Commits



```
feat: agrega búsqueda
por correo
```

imperativo + contexto

3 Pull Requests



- ? qué cambió
- ? por qué
- ▶ cómo probarlo

4 Repositorio

```
README.md
.gitignore
LICENSE
CONTRIBUTING.md
```

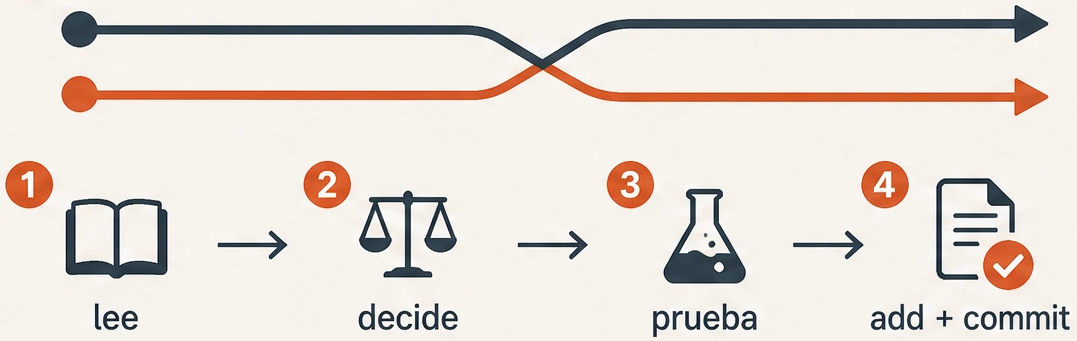
Las convenciones hacen que el equipo lea el proyecto con menos esfuerzo

De los errores a la receta de bolsillo

Si algo sale mal

`git log --oneline --graph`	inspecciona
`git diff`	compara
`git restore archivo.js`	descarta cambios no confirmados
`git revert ID_DEL_COMMIT`	revierte con otro commit

Conflicto de merge



Receta de bolsillo

```
git pull origin main
git switch -c feature/nueva-tarea
# trabajar
git status
git add .
git commit -m "feat: describe el cambio"
git push -u origin feature/nueva-tarea
```

Pull Request → revisión → merge

Trabaja localmente • confirma unidades pequeñas • sincroniza con frecuencia • protege main